

2c2025 - 2do Parcial - Ingeniería de Software I - FIUBA

Nuestro cliente quiere desarrollar una aplicación sencilla que permita gestionar historias de usuario utilizando un tablero Kanban simplificado. En este tablero, las historias de usuario pasan por diferentes estados o columnas: "Pendiente", "En Progreso", y "Finalizada".

Cada historia de usuario tiene un nombre, una prioridad (Alta, Media o Baja) y uno o más responsables. Las mismas comienzan en estado "Pendiente".

Se requiere que la aplicación provea las siguientes funcionalidades.

Administrar historias

- Crear una nueva historia de usuario indicando su nombre, prioridad y uno o más responsables. Si no se define prioridad, por defecto es Media. Los responsables son opcionales pero se debe definir un nombre válido para la historia de forma obligatoria.
- Agregar uno o más responsables a una historia.
- Los desarrolladores pueden mover una historia de usuario entre los distintos estados según las siguientes reglas:
 - De "Pendiente" solo puede pasar a "En Progreso". En este caso, se debe asignar al menos un responsable previamente.
 - De "En Progreso" puede pasar tanto a "Pendiente" (si se detuvo temporalmente) como a "Finalizada".
 - De "Finalizada" no puede moverse a otro estado.
 - No se puede mover desde un estado al mismo estado (Ej: De "Pendiente" a "Pendiente").

NO incluido en esta primera versión:

- Quitar responsables a una historia.
- Cambiar la prioridad o cambiar el nombre de una historia.

Ejemplos:

- Si se crea una historia de "Implementar login" con prioridad Alta asignada a "Ana", la misma inicialmente estará en el estado "Pendiente".
- Al mover la historia a "En Progreso" y luego intentar moverla directamente a "Pendiente" nuevamente es posible.
- Una historia "Desplegar servidor" con prioridad Media asignada a "Carlos" puede moverse de "Pendiente" a "En Progreso" y posteriormente a "Finalizada", quedando fija en este último estado.

Permisos según roles

- Además de los desarrolladores, otros miembros del proyecto pueden mover las historias entre las distintas columnas del tablero. Notar que para mover una historia no es necesario estar asignado como responsable de la misma. Las reglas para ellos son las mismas que las mencionadas arriba, con algunas excepciones, listadas a continuación:
 - El product owner (un rol más dentro del proyecto) puede mover historias de estado "Finalizada" a "En Progreso" o incluso a "Pendiente". Esto es útil para los casos donde el PO considera que las historias no cumplen el criterio de Done.
 - Por su parte, un miembro con rol de QA puede mover desde "Finalizada" a "En progreso" para los casos donde detecte fallos en las pruebas manuales de historias ya consideradas como terminadas por los desarrolladores responsables.

Equipos de trabajo como responsables

- Alternativamente, las historias de usuario también pueden tener como responsable a uno o más equipos de trabajo (en vez de miembros del proyecto individuales). Dos o más miembros del proyecto pueden formar equipos de trabajo, asignándoles un nombre. Por ejemplo, se debe poder asignar a la dupla "Juana-Pedro" que suele hacer pair programming a una o más historias.
- Un equipo de trabajo puede estar formado también por otros equipos o miembros. Por ejemplo, puede haber una historia que sea implementada por un equipo conformado por una pareja de desarrolladores ("subequipo") y un QA. También puede haber una pareja de desarrolladores que realizan tareas de backend y otra pareja que implementa tareas de frontend, para una misma historia.
- Se debe poder consultar si una historia tiene un determinado miembro asignado a partir de su nombre (independientemente de que si es el nombre de un equipo, el nombre de un miembro forme parte de un equipo o un miembro individual)

NO incluido en esta primera versión:

- Administrar un equipo de trabajo (agregar/quitar miembros, cambiarle el nombre, etc).
- Validación de consistencia en la creación de equipos de trabajo (ej: que sean al menos 2 miembros, que no se repitan individuos en subequipos, etc)

Extensibilidad

El cliente indicó que posiblemente agregue más estados y reglas en el futuro, como por ejemplo:

- Nuevos estados ("Revisión", "Bloqueado", etc.).
- Condiciones más específicas para movimientos entre estados (por ejemplo, "no puede ir a Finalizada sin revisión previa").

- Nuevos roles dentro del proyecto (además de desarrollador, *product owner* y QA) que permiten flexibilidad sobre las reglas de pasaje de estado (como el caso del product owner).
- Nuevas responsabilidades para los equipos de trabajo.

El modelo debe permitir estas modificaciones futuras sin cambios significativos.

Tips / Ayudas

- Recordar que se puede definir la = de cualquier objeto implementando el mensaje “=”. Tener en cuenta que para poder verificar que un objeto pertenece a un conjunto, además del “=”, se debe definir el mensaje **hash**. Este último se puede implementar enviando “hash” a un colaborador interno que sea representativo de la identidad del objeto.
- Quitar código repetido en los tests de forma temprana, en particular cómo se crean los objetos de prueba, puede ayudarles mucho para luego hacer refactors ;)

Modalidad de trabajo

La tarea se debe realizar mediante TDD, siguiendo las heurísticas de diseño vistas durante la cursada. En el modelo final deben pasar todos los tests desarrollados.

Realizar un desarrollo iterativo e incremental. Recordar que es más importante la calidad que la cantidad. Es decir, terminar $\frac{2}{3}$ del parcial con un buen modelo puede suponer mucho más nota que terminarlo completo a nivel funcional pero con un diseño pobre.

Entrega

- Entregar el fileout de la categoría de clase “**IS1-2c2025-2doParcial**” (deben ponerle ese nombre!), que debe incluir toda la solución (modelo y tests). El archivo de fileout se debe llamar: **2c2025-1erParcial.st**
- Entregar también el archivo que se llama **CuisUniversity-nnnn.user.changes**
- Probar que el archivo generado en 1) se cargue correctamente en una imagen “limpia” (o sea, sin la solución que crearon. Usen otra instalación de CuisUniversity/imagen si es necesario) y que todo funcione correctamente. Esto es fundamental para que no haya problemas de que falten clases/métodos/objetos en la entrega.
- Deben realizar la entrega enviando mail a: **fiuba-ingsoft1-doc@googlegroups.com** con el Subject: **Padrón NNNNN - Solución 2do parcial 2c2025.**
- **NO comprimir** los adjuntos (a no ser que sea estrictamente necesario).

RECOMENDACIÓN IMPORTANTE: Salvar la imagen de manera frecuente o con el autosave.

Se asume que a esta altura de la cursada saben trabajar con la imagen, recuperarla, recuperar código fuente, revertir cambios y demás incidencias que pudieran ocurrir durante el examen. Revisen bien los puntos de arriba. Cualquier error en los nombres o formato podrían ser penalizados en la nota.

IMPORTANTE: No retirarse sin tener el OK de los docentes de haber recibido la resolución por algún medio.

CERRAR EL TRABAJO A LAS 21:50.

LAS ENTREGAS RECIBIDAS DESPUÉS DE LAS 22:00 HRS NO SERÁN TENIDAS EN CUENTA